

二分查找

一个猜数的游戏，假设有一个人要我们猜 1-100 之间的一个数。那么最好的方法就是从 1-100 的中间数 50 开始猜。如果要猜的数小于 50，就猜 25（1-49 的中间数）；如果要猜的数大于 50，就猜 75（51-100 的中间数）。重复这个过程来缩小猜测的范围，直到猜出正确的数字。二分查找的工作方法类似于此。

二分查找也称折半查找（Binary Search），它是一种效率较高的查找方法。同时，二分查找还要求线性表必须采用顺序存储结构，而且表中元素按关键字有序排列。

二分查找操作的数据集是一个有序的数据集。开始时，先找出有序集合中间的那个元素。如果此元素比要查找的元素大，就接着在较小的一个半区进行查找；反之，如果此元素比要找的元素小，就在较大的一个半区进行查找。在每个更小的数据集中重复这个查找过程，直到找到要查找的元素或者数据集不能再分割。

二分查找能应用于任何类型的数据，只要能将这些数据按照某种规则进行排序。然而，正因为它依赖于一个有序的集合，这使得它在处理那些频繁插入和删除操作的数据集时不太高效。这是因为，对于插入和操作来说，为了保证查找过程正常进行，必须保证数据集始终有序。相对于查找来说，维护一个有序数据集的代价更高。此外，元素必须存储在连续的空间中。因此，当待搜索的集合是相对静态的数据集时，此时使用二分查找是最好的选择。

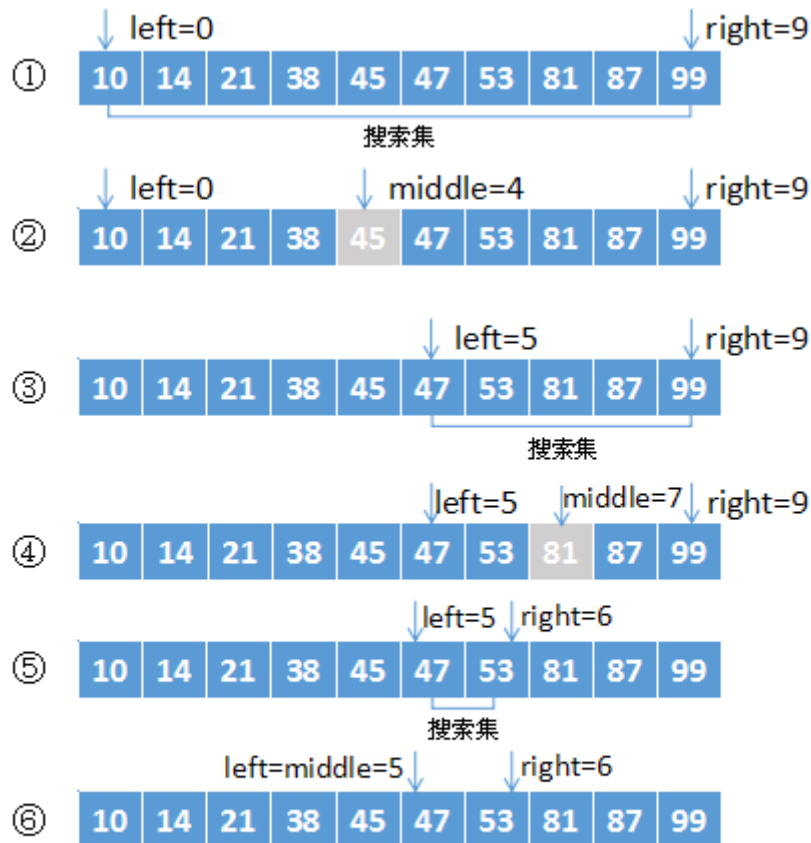


图:用二分查找搜索47

以下是简单的二分代码：

```
1. for (i=1;i<=n;i++) scanf("%d",&a[i]);
2. l=1,r=n;
3. while (l<r)
4. { mid=(l+r)>>1;
5.   if (x==a[mid]) break;
6.   if (x<a[mid]) r=mid-1; else l=mid+1;
7. }
8. printf("%d\n",mid);
```

二分查找 (BS,1s, 128MB)

【题目描述】

有 N 个数倒序排列，求出 x 是其中第几个数；若 x 不存在这 N 个数中，则输出 “Yah that’s a stupid ice elf!”

【输入文件】BS. in

输入第 1 行用空格隔开的两个数 N , x ;
第 2 行总共 N 个数，倒序排列。

【输出文件】BS. out

输出 1 行，即 x 是第几个数；若找不到，则输出 “Yah that’s a stupid ice elf!”。

【样例输入】

```
5 3
10 7 3 1 -1
```

【样例输出】

```
3
```

【数据规模】

对于 50% 的数据： $1 \leq N \leq 1000$;
对于 100% 的数据： $1 \leq N \leq 100000$ ；保证其余数据在 `int` 范围内；

浅谈二分的边界问题：

<https://www.luogu.com.cn/blog/HOJQVFNA/qian-tan-er-fen-di-bian-jie-wen-ti>

二分答案

我们知道，二分查找是在一个有序的数据集中，利用其数据元素单调递增/递减的特性，对符合“值相等”这个条件的数进行二分查找。这个二分查找的前提就是要查找的数据集符合单调性，且查找的数据元素又满足一定条件。

假如有这么一类问题：获得其答案需要付出相应的代价，而且付出代价越高，答案的值就越好，若要获取一个最优答案，且该答案所需付出的代价满足一定的范围条件，则这类题目可用二分法求解。

因为代价越高，答案越好，说明代价与答案之间的关系呈单调性；同时，答案所对应的的代价要满足一定范围，即答案“需要满足一定条件”。那么，我们就可以二分答案，来找到最优却又符合条件的结果。

砍树 (Cutt,1s, 128MB)

【题目描述】

伐木工人米尔科需要砍倒 M 米长的木材。这是一个对米尔科来说很容易的工作，因为他有一个漂亮的新伐木机，可以像野火一样砍倒森林。

米尔科的伐木机工作过程如下：米尔科设置一个高度参数 H （米），伐木机升起一个巨大的锯片到高度 H ，并锯掉所有的树比 H 高的部分（当然，树木不高于 H 米的部分保持不变）。米尔科就得到树木被锯下的部分。

例如，如果一行树的高度分别为 20，15，10 和 17，米尔科把锯片升到 15 米的高度，切割后树木剩下的高度将是 15，15，10 和 15，而米尔科将从第 1 棵树得到 5 米，从第 4 棵树得到 2 米，共得到 7 米木材。

米尔科非常关注生态保护，所以他不会砍掉过多的木材。这正是他为什么尽可能高地设定伐木机锯片的原因。帮助米尔科找到伐木机锯片的最大的整数高度 H ，使得他能得到木材至少为 M 米。换句话说，如果再升高 1 米，则他将得不到 M 米木材。

【输入文件】Cutt.in

输入第 1 行：2 个整数 N 和 M ， N 表示树木的数量， M 表示需要的木材总长度。

第 2 行： N 个整数表示每棵树的高度，值均不超过 1000000000。所有木材长度之和大于 M ，因此必有解。

【输出文件】Cutt.out

输出第 1 行：1 个整数，表示砍树的最高高度。

【样例输入】

```
5 20
4 42 40 26 46
```

【样例输出】

【数据规模】

对于 30%的数据保证： $N \leq 100$ ；

对于 100%的数据保证： $N \leq 10^6$ ， $M \leq 2 \times 10^9$ 。

三分法

前面提到二分的一个重要前提就是，答案存在于一个单调有序的序列中，反之，若序列不单调，则二分法难以为继。

一个简单的例子，若在处理数学函数的极值问题时，存在一个单峰函数，若不采用导数求解 0 值的方式，二分法就不可用。此时可以在区间内用两个 `mid` 将区间分成三份，这样的查找算法称为三分法。三分分法很多，按情况和喜好来吧。

【模板】三分法 (YML,1s, 128MB)

【题目描述】

如题，给出一个 N 次函数，保证在范围 $[1, r]$ 内存在一点 x ，使得 $[1, x]$ 上单调增， $[x, r]$ 上单调减。试求出 x 的值。

【输入文件】YML.in

输入第一行一次包含一个正整数 N 和两个实数 l, r ，含义如题目描述所示。
第二行包含 $N+1$ 个实数，从高到低依次表示该 N 次函数各项的系数。

【输出文件】YML.out

输出为一行，包含一个实数，即为 x 的值。若你的答案与标准答案的相对或绝对误差不超过 10^{-5} 则算正确。

【样例输入】

```
3 -0.9981 0.5
1 -3 -3 1
```

【样例输出】

```
-0.41421
```

【数据规模】

对于 100% 的数据保证： $6 \leq N \leq 13$ ，函数系数均在 $[-100, 100]$ 内且至多 15 位小数， $|l|, |r| \leq 10$ 且至多 15 位小数。 $l \leq r$ 。